

Real-Time Obstacle Avoidance for Mobile Robots using Deep Reinforcement Learning and Model Predictive Control

Shao-Hsuan Huang, Valdemar Samuelsson, Mathanesh Vellingiri Ramasamy, Rishikesh Vishnu Sivakumar, and Yu Kang

Abstract—Safe, efficient and collision-free navigation is crucial for autonomous mobile robots (AMRs) in industrial automation for various logistics and material handling applications. Previous studies have shown the potential of the hybrid models incorporating Model Predictive Control (MPC) and Deep Reinforcement Learning (DRL) in improving the performance of the mobile robots. However, there are certain challenges in computational complexity and stability. In this project, we propose three approaches to enhance DRL-MPC integration: (1) incorporating Q-functions into MPC’s cost function, (2) embedding MPC within the DRL learning process, and (3) generating reference paths using DRL predictions to enhance the computational complexity and control performance. The simulated system showed significant improvements in terms of success rate, computational time, and time-to-destination, providing a robust solution for the navigation of autonomous mobile robots.

I. INTRODUCTION

AMRs play a vital role in the transformation of industrial operations by enabling efficient material handling systems. They provide logistic support by navigating as a fleet in a complex and dynamic environment, ensuring operational safety and productivity. There are certain challenges when it comes to trajectory planning, obstacle avoidance and decision making in scenarios with multiple robots and dynamic obstacles. In such complex scenarios, traditional navigation algorithms such as rule-based and optimization methods cannot be relied upon since they fall short in handling non-convex obstacles and dynamically complex environments.

MPC is a more stable and robust control strategy where we solve the optimization problem at each time step over a certain period of time (time horizon). But the problem with the MPC is its requirement constraints when it comes to a more complex environment. On the other hand, DRL is widely adopted for its adaptive ability to learn optimal policies in complex environments. However, its reliance on excessive training data and lack of inherent stability mechanisms restrict its usage in real-time scenarios. Therefore, previous work on this project used a hybrid approach by integrating DRL and MPC frameworks in order to. The existing Hybrid Deep Deterministic Policy Gradient (HYB-DDPG) method in [2] shows a potential improvement in trajectory planning and obstacle avoidance by using DRL to generate a reference trajectory for the MPC when the original reference is blocked by an obstacle. This is done through a switch, which decides if the MPC should run on the global reference or one generated by DRL. However, there are still some challenges in computational efficiency, performance and robustness. This project aims to enhance the existing DRL-MPC framework by eliminating the reliance on the switching mechanism in order to achieve a more unified

system with improved computational efficiency and control performance.

Some of the ideas and evaluation criteria considered are given below:

- 1) **Q-function Integration:** Incorporate DRL’s Q-functions into MPC’s optimization process, replacing the switching mechanism.
- 2) **Action Space Optimization:** Refine DRL’s action space to align with MPC’s control objectives, reducing computational overhead.
- 3) **Stability and Feasibility Checks:** Ensure control actions generated by DRL meet MPC’s stability and feasibility criteria.

II. PRELIMINARIES

A. Model Predictive Control

MPC is a widely used control strategy that computes control actions by solving an optimization problem at each time step, for a finite predictive horizon. It predicts the future states of the system based on a model and then optimizes the control inputs to minimize a cost function, while satisfying the system’s state and control input constraints, making it an efficient and more stable control strategy for a constrained optimization problem. Consider a discrete-time system with the motion model:

$$x_{k+1} = f(x_k, u_k) \quad (1)$$

where x_k is the state at time step k , u_k is the control input at time step k , and $f(x_k, u_k)$ represents the system dynamics that describes how the state evolves based on the current state and control input. Let $z_k = [x_k, y_k]^T$ be a subset of the state vector x_k that represents the planar position of the robot, where x_k and y_k are the position coordinates at time step k . $\tilde{x}_k$ and $\tilde{u}_k$ represents the desired behaviour of the system. $\bar{x}$ is the initial state of the robot while U_k is the constraint on the control input at each time step. Now, we define the constrained optimal control problem (OCP) over a finite horizon N , with an objective to minimize the error from the reference trajectory while satisfying the control constraints and system dynamics. From [4], the OCP can be formulated as:

$$\min_{u_0:N-1} J_N + \sum_{k=0}^{N-1} (\|x_k - \bar{x}_k\|_{Q_x}^2 + \|u_k - \bar{u}_k\|_{Q_u}^2) \quad (2)$$

with the following constraints:

$$\text{s.t.} \quad x_0 = \bar{x} \quad (3)$$

$$x_{k+1} = f(x_k, u_k), \quad k \in \mathbb{N}_{[0, N-1]} \quad (4)$$

$$u_k \in U_k, \quad k \in \mathbb{N}_{[0, N-1]} \quad (5)$$

$$z_k \notin \mathcal{O} \cup \mathcal{D}_k, \quad k \in \mathbb{N}_{[0, N]} \quad (6)$$

where J_N is the terminal cost, Q_x and Q_u are the penalty weights of the state and control input, $\mathcal{O}$ and $\mathcal{D}_k$ are the static and dynamic obstacle areas that the robot should avoid, and $\mathbb{N}[a, b]$ denotes the integer set from a to b , representing the time steps within the predictive horizon. In the case of multi-robot scenarios, other robots can be perceived as dynamic obstacles for the ego robot, satisfying the collision avoidance constraint.

MPC solves the optimization problem with the Receding Horizon Control (RHC) approach where the optimization problem is solved at each time step using the current states of the system, while only applying the first control input. Once the first control action is executed, then the process is repeated with the new updated state measurements. This ensures that the robot adapts well to the changes in the environment and the dynamics of the system, since we reformulate the optimization problem by considering the current state and updated reference trajectory at each time step.

However, in the real-time scenarios, it is computationally demanding to solve this optimization problem, especially when the environment is complex or when there is a longer prediction horizon. This is due to the need for the system to handle potential obstacles and dynamics of the environment along with the system dynamics and state constraints. Thus, it is necessary to use efficient solvers and algorithms to make sure that the MPC can operate in an efficient manner for tasks involving such high-dimensional systems and faster dynamics.

B. Deep Reinforcement Learning

Combining the principles of Reinforcement Learning (RL) and Deep Learning (DL), DRL is used to tackle complex decision making tasks in a dynamic and high dimensional state and action spaces. It uses deep neural networks to perform approximation for policies and value functions. Some of its key components includes:

- Agent and Environment
- Policy
- Reward
- Value Function

Markov Decision Process (MDP) is a framework used for sequential decision making in reinforcement learning [5]. It is defined by:

- S : The set of states in the environment.
- A : The set of actions available to the agent.
- $P(s'|s, a)$: The state transition probability function.
- $R(s, a)$: The reward function (feedback for state-action pairs).

- $\gamma \in [0, 1]$: The discount factor (to balance immediate versus future rewards).

Here, the objective is to learn an optimal policy π^* that maximizes the discounted cumulative reward, defined as:

$$G_k = \sum_{i=0}^{\infty} \gamma^i R_{k+i+1}$$

C. Q-Learning and Deep Q-Networks

Q-Learning [7] is a classical RL algorithm that learns an action-value function $Q(s, a)$, representing the expected return for taking action a in state s . These values $Q(s, a)$ are updated in accordance with the rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a' \in A} Q(s', a') - Q(s, a) \right]$$

where α is the learning rate. Once updated, they are stored in a look-up table. It is not feasible to store $Q(s, a)$ in a table for high-dimensional state spaces. To address this, Deep Q-Networks (DQNs) are used to remove this limitation by employing deep neural networks to approximate $Q(s, a; \theta)$ where θ represents the networks parameters. DQNs [6] introduce techniques such as experience replay and target networks to stabilize training and improve convergence.

D. Policy Gradient Methods and Actor-Critic Approaches

Policy gradient methods [5] aim to learn a parameterized policy $\pi(s|\theta^\pi)$ directly, optimizing the objective function (loss). On the other hand, Actor-critic methods [5] combine the advantages of value-based and policy-based approaches. The actor updates the policy parameters to improve action selection, while the critic evaluates the policy by estimating a value function. The actor and critic are updated by minimizing the loss as follows:

$$\nabla_{\theta^\pi} L = \mathbb{E} \left[\nabla_a Q(s_k, a | \theta^Q) \Big|_{a=\pi(s_k)} \nabla_{\theta^\pi} \pi(s_k | \theta^\pi) \right]$$

$$\mathcal{L}(\theta^Q) = \mathbb{E} \left[(Q(s_k, a_k | \theta^Q) - \bar{G}_k)^2 \right]$$

$$\bar{G}_k = R(s_k, a_k) + \gamma Q(s_{k+1}, \pi(s_{k+1} | \theta^\pi) | \theta^Q)$$

Deep Deterministic Policy Gradient (DDPG) [6] is a notable actor-critic algorithm designed for continuous action spaces, employing neural networks for both the actor and critic components. Even with all the remarkable success in the use of DRL methodology across domains including robotics and autonomous vehicles, it is still a challenging task in terms of managing the sample inefficiency, exploration-exploitation trade-offs, and stability issues during training.

III. METHODOLOGY

Three different approaches were tried, each dealing with different aspects of the problem. The first method incorporates the reward function of the DRL-agent into the MPC problem. The second method involves taking the MPC calculation into the DRL learning process. Finally, the last method attempts to improve the MPC reference trajectory using DRL. All of them are trying to optimize the outcome of the hybrid MPC-DRL model and are evaluated with the same several map scenarios that will be introduced in more detail in the Evaluation section. We illustrate the details of each method below:

A. *Q-function as final cost for Model Predictive Control:*

In this method, the existing final cost of the MPC is discarded and replaced with the learned Q-function. The Q function can be obtained from one of the two already existing deep reinforcement learning (DRL) models, Deep Deterministic Policy Gradient (DDPG) and Twin Delayed Deep Deterministic Policy Gradient (TD3). As the final cost, the Q-function represents the expected cumulative reward for the final state-action pair.

This approach offers several benefits, including enhanced long-term decision-making, as the MPC can now make decisions that are optimal over a longer time horizon beyond its prediction window. Additionally, it enables adaptive behavior, as the learned Q-function allows the controller to adapt to situations not explicitly modeled in the MPC, potentially improving performance in complex or uncertain environments.

To integrate the Q-function with MPC's optimization framework, we developed a lookup table that creates a continuous approximation of the Q-function. We start off with discretizing the state-action space by constructing a grid of points that spans the operational region. This discretization is applied across all state dimensions (position coordinates and orientation) and action dimensions (linear and angular velocities), ensuring coverage of all feasible ranges. Using the trained critic network from our DRL model, we compute Q-values for each state-action combination in this grid. This computation generates a discrete mapping that associates each state-action pair with its corresponding Q-value, effectively capturing the learned value function across the discretized space. The process concludes with normalization using the maximum Q-value from our samples, followed by CasADi's bspline interpolation to transform the discrete mapping into a continuous function. This interpolation enables smooth gradient information necessary for the MPC optimization while maintaining computational efficiency.

The terminal cost in MPC is modified to include the Q-function contribution:

$$J_{terminal} = q_w \cdot (Q_{max} - Q_{ip}(s_H, a_H, t))^2 \quad (7)$$

The weight parameter q_w balances the Q-function's influence with other cost terms. This formulation encourages the MPC to select terminal states with high long-term value.

The integration of the Q-function through this lookup table method provides a practical way to combine the long-term value information learned through DRL with MPC's constraint handling and optimization capabilities.

B. *Including MPC within the training process of DRL:*

In previous work, the DRL-agent was trained without knowing the planned route of MPC. This makes the action during training affect the performance differently than in the hybrid model. Therefore, incorporating the MPC into the training process would give rewards to the agent more accurately of the hybrid models performance. That is, the original action prediction of the DRL model will be further calculated as reference path and fed into the MPC model (Same procedure as the previous MPC-DRL hybrid model). The computational result of the MPC model will be considered as reward for the DRL training process, including solver time, the cost of the MPC model and the penalty of the model. The DRL model, considering these costs and rewards, will generate a prediction considering not to have high cost, penalty and less solving time of MPC, which is expected to further reduce the solving time of the whole hybrid process while maintaining stability. Also, since the MPC is used inside the training and prediction process, the switch will be used during training.

The training process, however, becomes very complex when including the MPC solver, which makes the tuning and training process difficult. The tuning parameters are hard to design since the cost and penalty of MPC is around 1000 or more, but the reward of the DRL model is around 5, which makes the two models hardly align. Even if we normalize the cost and penalty, the overall process still has difficulty to converge and remain at a high reward. Adding with the complexity of the whole training process, it makes the whole result even hard to stabilize. It can be observed in the results that the result is not improved with 10,000 time steps trained.

C. *Generating reference path using DRL*

In the previous Hybrid model, when the reference path is blocked by an obstacle a new temporary reference path is generated using one action of the DRL model and then continuing with the generated velocity and angular velocity. In this part a new hybrid model DRL-reference-path is created where the reference trajectory is generated using multiple DRL actions, the same number as the MPC horizon length. This removes the need of switching between two different references as it can use the same reference trajectory everywhere. To generate the reference trajectory, the DRL-agent takes steps in a simulated environment without moving the obstacles. This results in a model that moves similarly to the DRL model, but with MPC applied to smooth the velocity and angular velocity, while also preventing the agent from colliding with obstacles. The drawback of this approach is that the DRL action needs to be calculated for each step in the MPC horizon which creates more DRL calculations. Using a reference trajectory from multiple DRL actions reduces the risk of the reference trajectory going through an obstacle and that can reduce the

MPC computation time in some cases. It can also improve the efficiency of the reference trajectory as it doesn't need to follow the global reference when there is no obstacle.

IV. EVALUATION

A. Evaluation Metrics

The Model's performance will be evaluated using the following metrics [2]:

1) **Computational Time:** Time taken to generate control actions. Recorded are the mean, maximum, and standard deviation of computation times across models, aiming for lower values.

2) **Deviation:** The mean and maximum distances from the reference path are measured, with smaller deviations indicating better accuracy.

3) **Action smoothness:** Smoothness is calculated as the mean of the approximated jerk of the AMR's speed and angular speed. Lower values indicate smoother actions, which are preferable for practical AMR applications.

4) **Finish time step:** The number of steps needed to complete the evaluation. Consistency across models is desired, as large variances suggest significant deviations from the intended speed.

5) **Success Rate:** Ratio percentage between the number of successful runs (robot reaching the goal without any collision) and the total number of runs.

B. Evaluation of Q-function as final cost for MPC method (Method 1):

The performance of the modified MPC model was evaluated across all maps for 50 trials. The results are compared with other models and compiled into a table.

1) **Computational Time:** The overall computational time increases to about 10^5 to 10^6 comparing to 10^4 to 10^5 for pure MPC model and 10^3 to 10^4 for the Hybrid model. This is expected as we calculate the Q-value lookup table, which increases the overall computational time.

2) **Deviation:** The mean deviation from the reference path across all tests ranges from 0.74m to 0.87m, with maximum deviations between 1.52m and 1.96m. The largest deviations occur in scenarios with multiple obstacles (1.96m for staggered configuration), while single rectangular and U-shaped obstacles maintain more controlled deviations. This indicates the model's ability to handle various obstacle configurations while maintaining relatively consistent path-following behavior.

3) **Action smoothness:** The mean smoothness across all tests ranges from 0.07 to 0.15, with maximum values between 0.14 and 0.41. The smoothest performance is observed in simple rectangular and dynamic obstacle scenarios (0.07 mean), while the most challenging U-shaped obstacles result in higher jerk values (0.15 mean).

4) **Finish time step:** The overall finish time step also increases to about 120 steps comparing to 88.425 steps of pure MPC (not including maps that has zero success rate of scene of e-small and f-large) and 92.65 of the Hybrid model. The same reason as the computational time section that we are

considering the Q-value finding process, the model will try to switch between relying more on MPC or DRL, which makes the overall steps longer for the switching action.

5) **Success Rate:** Right now the Q-value consideration is generated with scene f-large, and we can see that the success rate is scene dependent since the success rate is lower on other scenarios of scene 1 but higher on scene f-large and e-small(similar scene as f-large). The success rate for f-large is 0.64 which is superior than both pure MPC model and the Hybrid model, which indicates that, by considering the Q-value, the model can be more adaptable with un-convex obstacle, though the model will become scene dependent.

C. Evaluation of including MPC within the learning process of DRL (Method 2):

The performance of this method is compared with the Hybrid MPC-DDPG model and the result is evaluated in the following sections:

1) **Computational Time:** The overall computational time becomes larger comparing to the Hybrid model, it might be because we are considering the whole MPC calculation inside each prediction step that makes the whole computational time grow with each step.

2) **Deviation & Action smoothness:** The deviation for this method has no big difference between the original Hybrid method, and it might indicate that by changing the MPC model consideration into the training and prediction process will not influence the deviation, and the same case appears for the action smoothness part.

3) **Finish time step:** The finish time step has no big difference with the Hybrid method for scene 1 but slightly larger for scene 2. The reason might be that pure MPC model has low stability for these scenes that might cause the calculation and consideration time longer, and the instability also influence the success rate. The scene 1-3-1 also has instability for pure MPC model but with less finish time step, but because of its low success rate, the samples are too few to be considered here.

4) **Success Rate:** The success rate of most of the scenarios stay the same as the Hybrid model. However, we can observe that for scenes that have low success rates with pure MPC model will have lower success rate than the Hybrid model. It might be because either the model relies too much on the MPC model for prediction that need further fine-tuning, or the instability of the model itself like we discussed before in the Methodology section that cause it to be likely to fail.

D. Evaluation of reference path generated by DRL (Method 3):

The performance of the DRL-reference-path model is compared to the Hybrid model and the DRL model. The evaluation was done on all provided evaluation maps for 50 iterations and then the results are combined into a table.

1) **Computational Time:** The mean computational time of the DRL-reference-path model more than doubled compared to the Hybrid model. The maximum computation time decreased compared to the hybrid model for a majority of the test cases. On some maps the hybrid model has a maximum computation time that is more than 2 times longer compared to DRL-reference-model.

2) **Deviation:** The DRL-reference-path model performed similarly to the Hybrid model.

3) **Action smoothness:** The mean smoothness is similar to that of the hybrid model for all test cases.

4) **Finish time step:** The DRL-reference-path finished most test cases in a lot fewer time steps compared to the hybrid model. This may be because the DRL-reference-path reacts to obstacles as soon as it can detect them, compared to the hybrid model which only reacted when obstacles were close enough to make it switch between the global reference and DRL. Another reason might be that the reference path of the hybrid model changes a lot between time steps causing it to zigzag while the DRL-reference-path have fewer changes.

5) **Success Rate:** The DRL-reference-path got 100% success rate on most of the test which is better than all the other models. This shows that compared to the DRL-reference-path model improves upon the hybrid model with reducing the amount of time steps, while keeping the same or higher success rate.

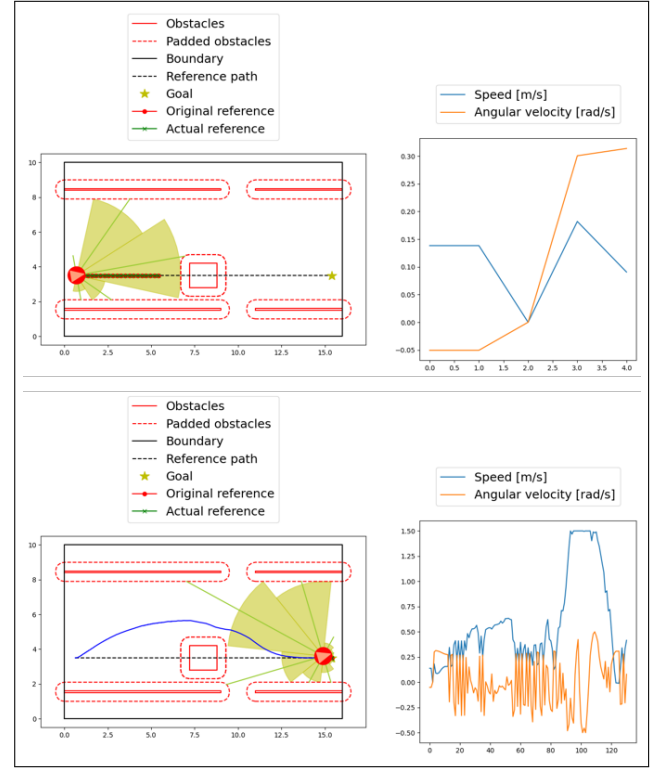


Fig. 2: Simulation result for Scene (1-1-2) using **HYB-DDPG (M1)**

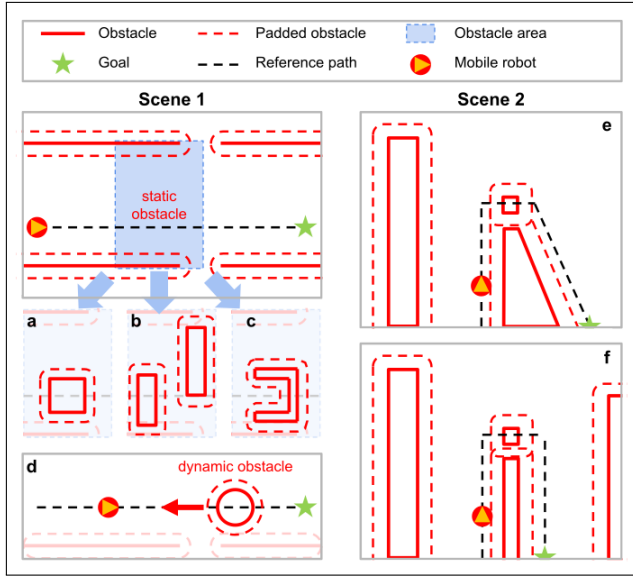


Fig. 1: Scenes for single-agent evaluation [1]. Scene 1 includes three types of static obstacles and one dynamic obstacle scenario. Scene 2 assesses the turning performance of AMRs with unexpected obstacles.

E. Results from different scenarios

Evaluation Results for different scenarios and methods can be seen in Fig. 4. Some of the simulation results can be seen in the Fig. 2 and Fig. 3.

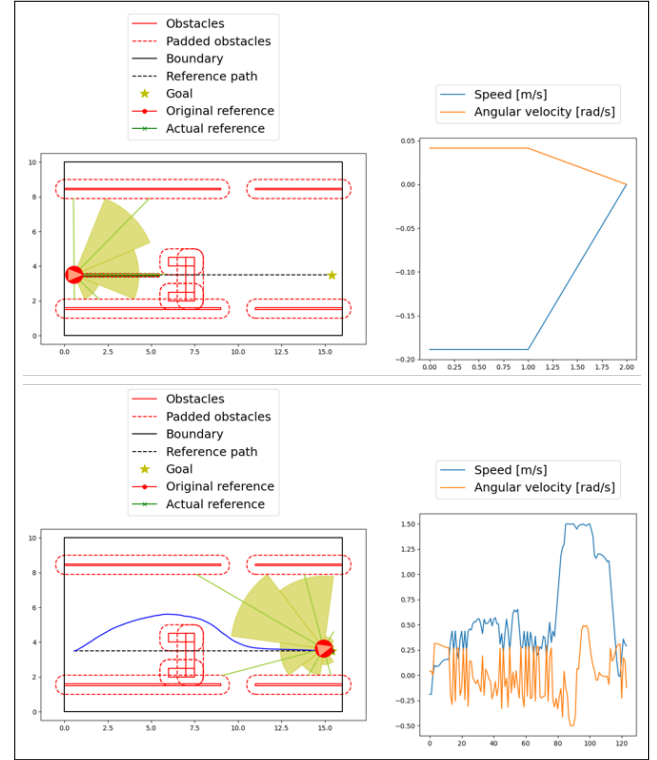


Fig. 3: Simulation result for Scene (1-3-2) using **HYB-DDPG (M1)**

Scene	Obstacle Type (Scene ID)	Method	Computation Time (ms/step)			Deviation (m)		Action smoothness		Other	
			mean	max	std	mean	max	speed	angular speed	finish time step	success rate (%)
Scene 1	(a) Rectangular obstacle (1-1-2)	MPC	245.85	1253.47	83054.14	0.39	1.49	0.04	0.06	69	94
		DDPG-L	0.33	0.72	0.01	0.84	1.75	0.09	0.2	58	100
		REF-DDPG-L	65.35	112.24	237.11	0.81	1.65	0.02	0.04	56	100
		HYB-DDPG (OG)	29.97	224.2	1527.18	0.84	2.29	0.02	0.06	80	100
		HYB-DDPG (M1)	1611.77	5081.67	1281916.35	0.87	1.7	0.07	0.18	113	94
		HYB-DDPG (M2)	112.95	897.04	26705.77	1	2.16	0.01	0.05	81	92
	(b) Two obstacles (1-2-1)	MPC	356.59	1259.07	83640.48	0.43	1.27	0.05	0.04	72	100
		DDPG-L	0.3	0.61	0.01	0.92	1.84	0.11	0.12	82	4
		REF-DDPG-L	81.76	181.8	606.51	0.71	1.64	0.04	0.04	80	100
		HYB-DDPG (OG)	38.78	150.82	866.08	0.82	2.14	0.04	0.05	98	100
		HYB-DDPG (M1)	1936.81	5026.06	1633470.2	0.74	1.6	0.12	0.27	165	2
		HYB-DDPG (M2)	174.27	633.61	15187.8	0.83	2.14	0.03	0.05	97	100
	(c) U-shape obstacle (1-3-2)	MPC	583.39	3989.42	1113825.28	0.14	0.3	0.15	0.2	-1	0
		DDPG-L	0.31	0.6	0.01	1.08	2.38	0.11	0.18	61	98
		REF-DDPG-L	72.52	109.06	234.7	0.95	1.99	0.02	0.04	58	100
		HYB-DDPG (OG)	73.47	842.56	25109.03	0.28	0.82	0.03	0.04	-1	0
		HYB-DDPG (M1)	2085.14	5031.06	1293683.37	0.77	1.71	0.15	0.4	98	64
		HYB-DDPG (M2)	86.9	1152.91	41671.47	0.3	0.85	0.03	0.04	87	2
Scene 2	(e) Sharp turn with an obstacle (2-1-2)	MPC	50.95	340.89	4043.25	0.12	0.56	0.04	0.07	147	8
		DDPG-L	0.31	0.76	0.01	0.92	2.03	0.09	0.18	120	100
		REF-DDPG-L	63.59	130.12	181.09	0.9	1.99	0.02	0.03	121	100
		HYB-DDPG (OG)	20.29	131.52	483.99	0.38	1.56	0.01	0.02	146	100
		HYB-DDPG (M1)	1207.57	5007.26	967383.62	0.55	3.75	0.01	0.01	-1	0
		HYB-DDPG (M2)	47.42	309.39	2000.97	1.01	2.54	0.01	0.05	186	66
	(f) U-turn with an obstacle (2-1-3)	MPC	135.55	295.76	4863.15	0.08	0.27	0.02	0.02	-1	0
		DDPG-L	0.32	0.83	0.01	0.73	1.48	0.12	0.18	122	92
		REF-DDPG-L	64.11	130.65	183.18	0.75	1.51	0.01	0.02	121	100
		HYB-DDPG (OG)	28.92	373.5	2030.02	0.41	1.45	0.02	0.02	145	100
		HYB-DDPG (M1)	1293.66	5043.25	1716473.7	0.48	3.31	0.04	0.03	-1	0
		HYB-DDPG (M2)	65.51	1189.7	11791.2	0.75	1.51	0.02	0.05	164	42

Fig. 4: Evaluation Results for different scenarios and methods.

V. CONCLUSION

In this project, different approaches to improve upon the existing model was tried. Using Q-learning as final cost for MPC showed promising results for a specific map by generating the Q-learning lookup table, but it suffered from long computation times and also the map-dependency that can only be stable on that specific map. If there is a way to find a general method to generate the lookup table for different map scenarios, it can reduce the overall stability and might be practical to have some improvement. Secondly, Incorporating MPC in the training loop did not improve upon the hybrid model. This may be because the MPC itself adds the complexity to the learning model, and the nonlinearity of the DRL is hard to cooperate with the linearity of the MPC model that makes the overall training process hard to reach convergence. Future work might be that we can try to start from simple model to try to find convergence before implementing the whole model to train it. Lastly, using multiple DRL-actions as the reference path for MPC showed promise, as it improved upon both finish time, success rate and maximum computation time. One drawback is that it has longer mean computation time. It needs to have a model of the nearby environment in order to generate this reference path, and that might not be possible in real world applications. The future work can try to find the balance between the horizon to compute the trajectory and the computational time to have a good performance with small computational time.

REFERENCES

- [1] Z. Zhang, Y. Cai, K. Ceder, A. Enliden, O. Eriksson, S. Kylander, R. Sridhara, and K. Åkesson, "Collision-free trajectory planning of mobile robots by integrating deep reinforcement learning and model predictive control," in *International Conference on Automation Science and Engineering (CASE)*. IEEE, 2023, pp. 1–7.
- [2] K. Ceder, Z. Zhang, A. Burman, I. Kuangaliyev, K. Mattsson, G. Nyman, A. Petersén, L. Wisell, and K. Åkesson, "Bird's-eye-view trajectory planning of multiple robots using continuous deep reinforcement learning and model predictive control," in *International Conference on Intelligent Robots and Systems (IROS)*, 2024.
- [3] N. Hansen, X. Wang, and H. Su, "Temporal difference learning for model predictive control," *arXiv preprint arXiv:2203.04955*, 2022.
- [4] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design (2nd Edition)*. Nob Hill Publishing, LLC, 2020.
- [5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018.
- [6] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, p. 529–533, 2015.
- [7] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, p. 279–292, 1992.
- [8] Z. Zhang, H. Hajieghrary, E. Dean, and K. Åkesson, "Prescient collision-free navigation of mobile robots with iterative multimodal motion prediction of dynamic obstacles," *Robotics and Automation Letters*, vol. 8, no. 9, pp. 5488–5495, 2023.
- [9] M. Kokot, D. Miklić, and T. Petrović, "A unified MPC design approach for AGV path following," in *International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 4789–4796.
- [10] A. Dahlin and Y. Karayiannidis, "Obstacle avoidance in dynamic environments via tunnel-following MPC with adaptive guiding vector fields," in *Conference on Decision and Control (CDC)*. IEEE, 2023, pp. 5784–5789.
- [11] H. Sun, W. Zhang, R. Yu, and Y. Zhang, "Motion planning for mobile robots — focusing on deep reinforcement learning: A systematic review," *IEEE Access*, vol. 9, pp. 69 061–69 081, 2021.
- [12] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, 2023.
- [13] T.-W. Weng, K. D. Dvijotham, J. Uesato, K. Xiao, S. Gowal, R. Stanforth, and P. Kohli, "Toward evaluating robustness of deep reinforcement learning with continuous control," in *International Conference on Learning Representations (ICLR)*, 2020.
- [14] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *International Conference on Learning Representations (ICLR)*, 2016.
- [15] H. Sang and S. Wang, "Motion planning of space robot obstacle avoidance based on DDPG algorithm," in *International Conference on Service Robotics (ICoSR)*, 2022, pp. 175–181.
- [16] A. Rodriguez-Ramos, C. Sampedro, H. Bavlé, I. G. Moreno, and P. Campoy, "A deep reinforcement learning technique for vision-based autonomous multirotor landing on a moving platform," in *International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 1010–1017.
- [17] C. Do, C. Gordillo, and W. Burgard, "Learning to pour using deep deterministic policy gradients," in *International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 3074–3079.
- [18] A. Sathya, P. Sopasakis, R. Van Parys, A. Themelis, G. Pipeleers, and P. Patrino, "Embedded nonlinear model predictive control for obstacle avoidance using PANOC," in *European Control Conference (ECC)*. IEEE, 2018, pp. 1523–1528.
- [19] J. Eschmann, *Reinforcement Learning Algorithms: Analysis and Applications*. Springer International Publishing, 2021, pp. 25–33.